Wireshark Traffic Analyzer: What App Are You Really Using?

EC441 Computer Networking

Apr 30th, 2026

Presented by Yena Yu

Yena Yu

EC441 A1

Content

1 Motivation

2 Why it matters

3 What I built

4 Features Used

5 Application Fingerprints

6 Demo Capture

7 Results : Traffic Patterns

8 Main Result: Flow Fingerprints

9 Takeaway + Limitations

Motivation

Modern Traffic Is Encrypted

- HTTPS / TLS / QUIC hide payload contents
- Deep packet inspection is less useful today
- But metadata is still visible:
  - packet sizes
  - timing
  - protocol (TCP/UDP)
  - ports
  - directionality

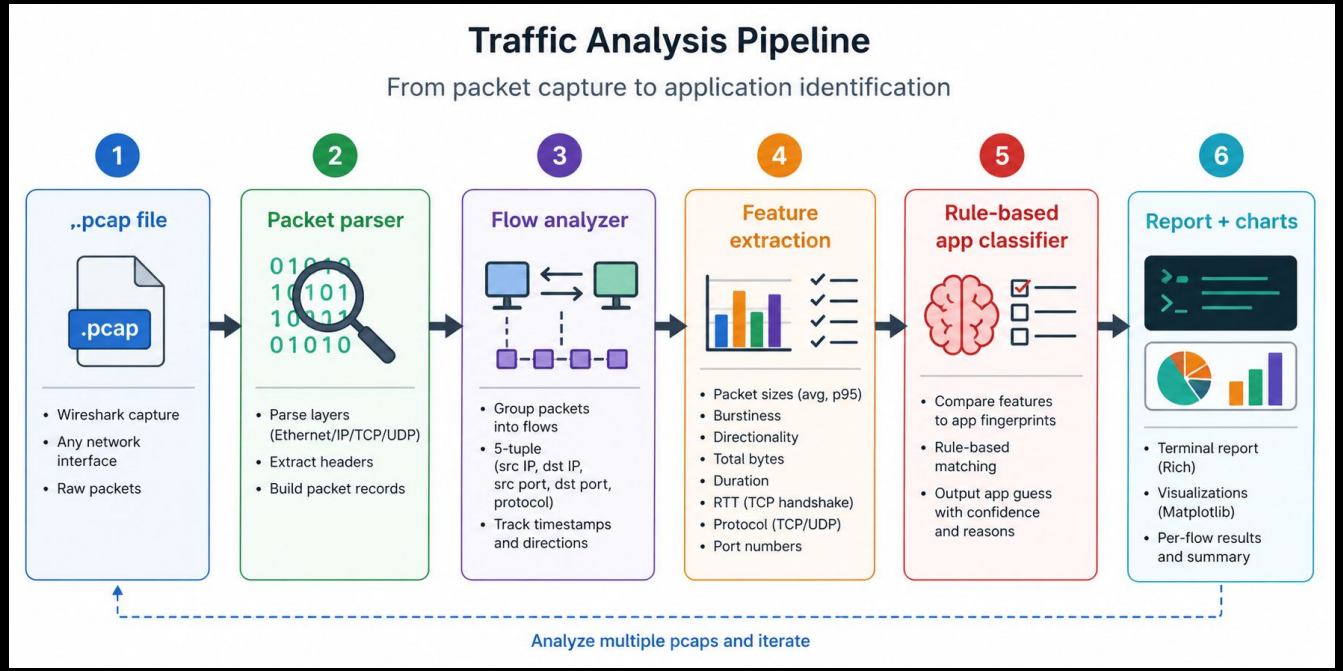
Key Idea

Encryption hides content, but not behavior.

Practical Uses

- Network observability
- Troubleshooting slow Wi-Fi
- QoS for real-time calls
- Security anomaly detection
- Privacy implications

What I Built

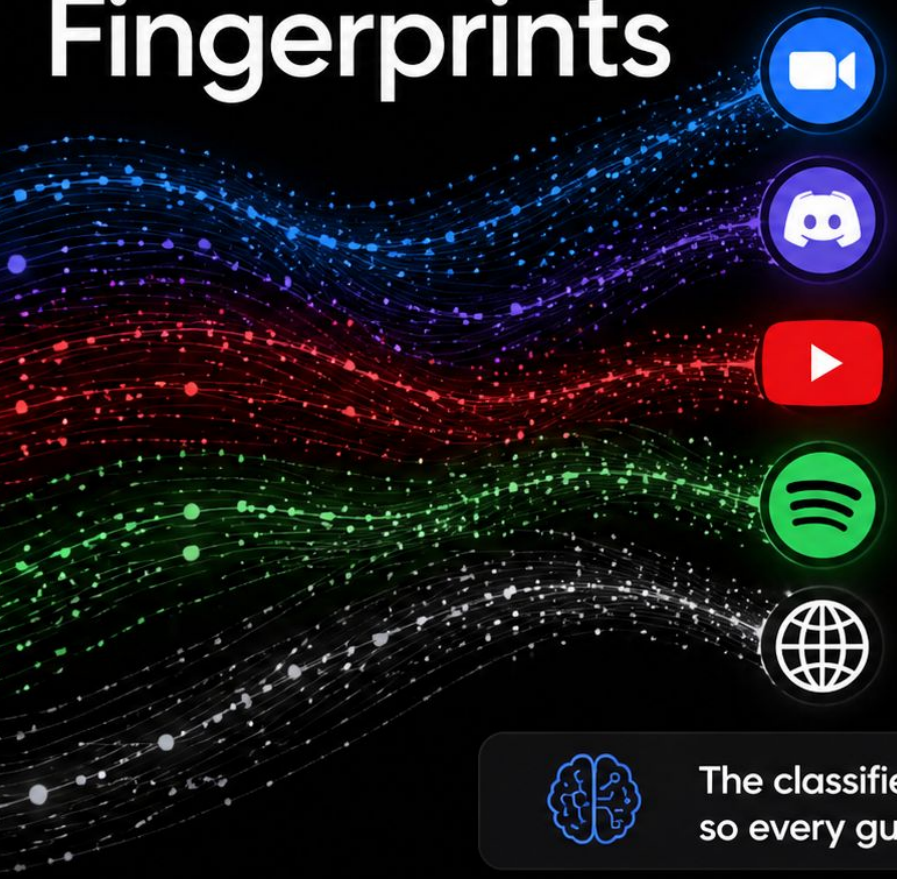

Traffic Analysis Pipeline
From packet capture to application identification
1
,.pcap file
.pcap
• Wireshark capture
• Any network interface
• Raw packets
2
Packet parser
01010
• Parse layers (Ethernet/IP/TCP/UDP)
• Extract headers
• Build packet records
3
Flow analyzer
• Group packets into flows
• 5-tuple (src IP, dst IP, src port, dst port, protocol)
• Track timestamps and directions
4
Feature extraction
• Packet sizes (avg, p95)
• Burstiness
• Directionality
• Total bytes
• Duration
• RTT (TCP handshake)
• Protocol (TCP/UDP)
• Port numbers
5
Rule-based app classifier
• Compare features to app fingerprints
• Rule-based matching
• Output app guess with confidence and reasons
6
Report + charts
• Terminal report (Rich)
• Visualizations (Matplotlib)
• Per-flow results and summary
Analyze multiple pcaps and iterate

Features Used

What The Analyzer Measures

Feature	Why it matters
TCP vs UDP	downloads vs realtime media
Ports	Zoom 8801, HTTPS 443, Discord voice
Packet size	voice small, video large
Burstiness	steady vs chunked traffic
Direction	two-way call vs download
RTT	TCP handshake latency

Application Fingerprints



Zoom Realtime Video	UDP, ports 8801–8810, bidirectional, realtime
Discord Voice Chat	UDP voice, small regular packets
YouTube Video Streaming	Large bursty TCP/443 video chunks
Spotify Audio Streaming	Smaller TCP/443 audio chunks
Web Browsing General Web Traffic	Many short TCP/443 flows

The classifier is rule-based, so every guess has an explanation.

Demo Capture

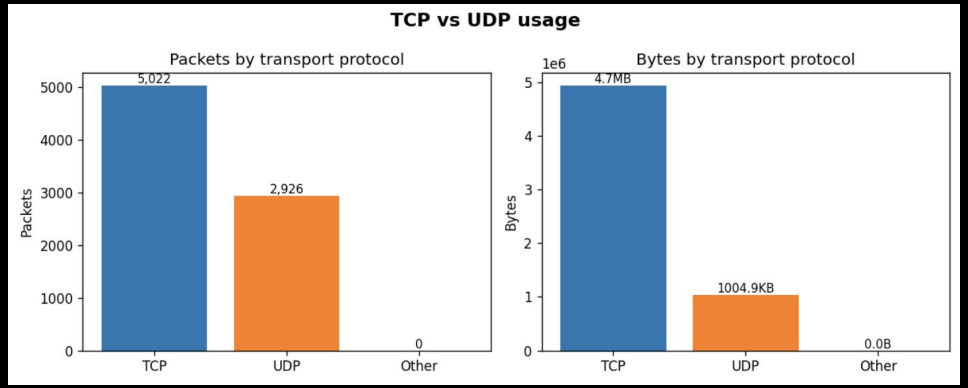
Five Apps at Once

- Synthetic .pcap generated with Scapy
- Works fully offline
- ~12 seconds of traffic
- ~8,000 packets
- ~5.7 MB total traffic
- Simulates:
  - Zoom
  - Discord
  - YouTube
  - Spotify
  - Web browsing

Generated synthetic pcaps so the demo works even if classroom Wi-Fi fails.

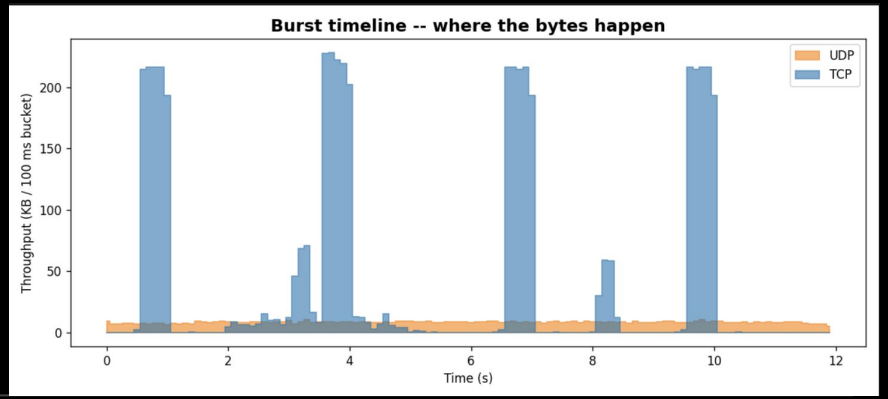
Results: Traffic Patterns

TCP/UDP + Burst Timeline

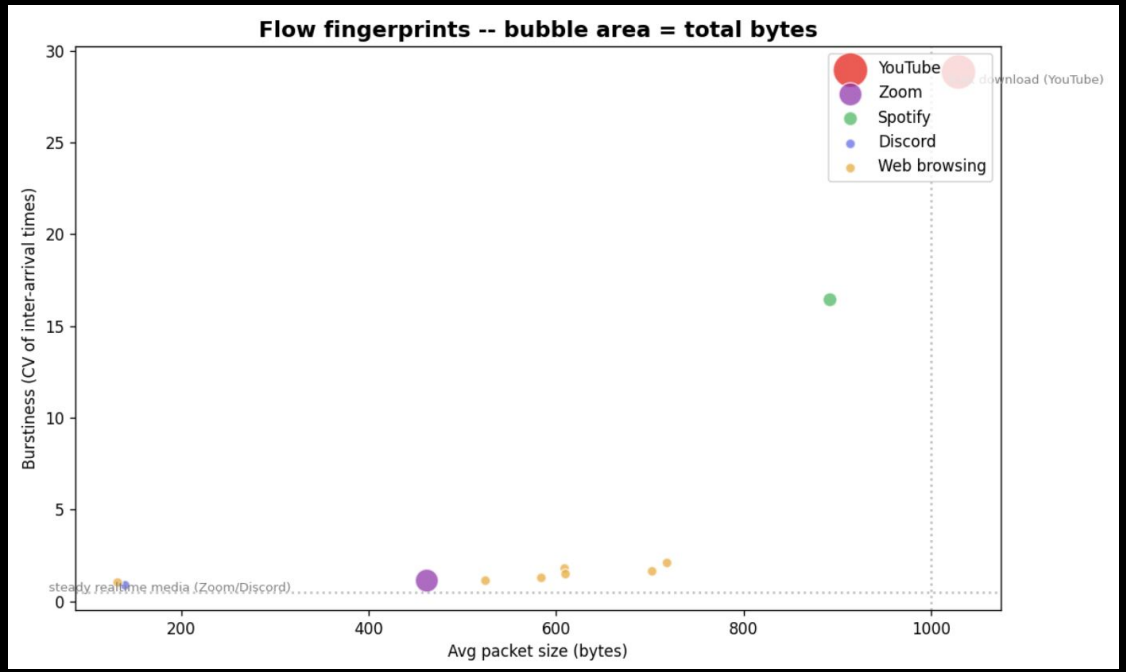


- TCP dominates bytes because YouTube / Spotify / web use HTTPS
- UDP represents realtime traffic like Zoom / Discord
- TCP appears in bursts
- UDP is steadier

Realtime apps are steady; streaming apps are bursty.



Main Result:
Flow Fingerprints



- x-axis = average packet size
- y-axis = burstiness
- bubble size = total bytes
- color = predicted app

The apps occupy different regions because their network behavior is different.

Takeaway + Limitations

What this demonstrates

1

Metadata can reveal app behavior even when content is encrypted

2 Useful for:

- debugging
- QoS
- security monitoring
- privacy analysis

3 Limitations

Synthetic dataset
Rule-based classifier
VPNs / padding / proxies can hide patterns
Many TCP/443 apps can look similar

4

Payload encryption protects content, but traffic shape still tells a story.

Thank you

Ready for what's next?

Let's talk

Yena Yu
Email: yenayu@bu.edu